



عملي

1

22

3300 sp

كلية الهندسة المعلوماتية

السنة الرابعة

Users and Groups in Linux

عملي مشترك



نظم التشغيل

RB Informatics; 28/10/2025

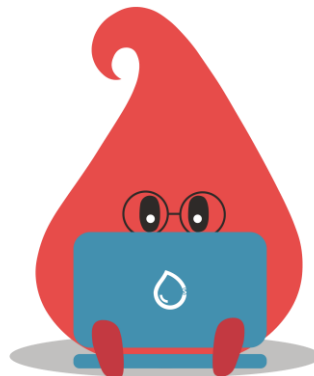
السلام عليكم ورحمة الله وبركاته

- تُعد نظم التشغيل من أهم ركائز علوم الحاسوب، فهي الجسر الذي يصل بين مكونات الجهاز المادية والبرمجيات.
- من خلالها نتعلم كيف تُدار الموارد، وتُنَفَّذ المهام، ويُوَفَّر للمستخدم بيئة عمل متكاملة.
- إن فهم نظم التشغيل هو الخطوة الأولى لفهم كيفية عمل الحواسيب الحديثة بكل تفاصيلها وتعقيداتها
- في هذه المحاضرة سنتعرّف عن أنظمة التشغيل وبالأخص نظام التشغيل linux وأوامره .

مقدمة عن نظام التشغيل Linux

ما هو نظام تشغيل Linux

نظام التشغيل هو البرنامج الذي يدير مباشرةً عتاد (مكونات) النظام وموارده مثل المعالج (CPU) والذاكرة والتخزين يقع نظام التشغيل بين التطبيقات والعتاد، ويقوم بإنشاء الاتصال بين جميع البرامج والموارد المادية التي تقوم بالعمل. يُعد لينكس نسخة مجانية ومفتوحة المصدر (Open Source) وهي مبنية من نظام يونكس (Unix)، تم تطويرها في الأصل بواسطة لينوس تورفالدز (Linus Torvalds)، الذي بدأ العمل على لينكس عام 1991.



■ الخصائص الرئيسية لنظام التشغيل Linux

1. مفتوح المصدر (open source): يمكن لأي شخص عرض ال source code وتعديله وتوزيعه.
2. آمن (secure): أقل عرضة للبرامج الضارة بفضل التحديثات المجتمعية و نظام الصلاحيات القوي (لا يعطي كامل الصلاحيات مباشرة) .
3. مستقر وفعال (stable & efficient): مثالي للخوادم (يمكن أن يعمل على الخوادم لفترة طويلة دون أن ينطفئ)، والأنظمة المدمجة، والأجهزة القديمة.
4. قابل للتخصيص (customizable): يمكنك الاختيار من بين توزيعات عديدة Ubuntu , Fedora , Arch ,Lubuntu وبيئات سطح مكتب متنوعة .
5. يدعم تعدد المستخدمين والمهام (multiusers & multitasking): يمكنه تشغيل عدة مستخدمين وعمليات في نفس الوقت.

■ مقارنة بين نظامي التشغيل Linux و Windows

Windows	Linux	وجه المقارنة
يتطلب عادةً ترخيصاً مدفوعاً.	مجاني ومفتوح المصدر (خياراً أكثر توفيراً).	التكلفة Cost
أكثر عرضة للبرامج الضارة والفيروسات بسبب انتشار النظام واعتماد المستخدمين على حسابات المدير وأكثر استهدافاً من قبل المخترقين لكثرة مستخدميه.	أكثر أماناً بفضل نظام الصلاحيات القوي وعدم منح صلاحيات الجذر (root) افتراضياً مما يقلل من خطر البرمجيات الخبيثة، المخترقون عادة لا يتوجهون لنظام لينكس لأنه أقل استخداماً.	الأمان Security
مغلق المصدر – الكود المصدري مملوك لشركة مايكروسوفت ولا يمكن تعديله.	يمكن لأي شخص فحص الكود وتعديله، مما يساهم في اكتشاف الثغرات بشكل أسرع.	الشفافية مفتوحة المصدر Open-source transparency
قد يواجه بطئاً أو أخطاءً بمرور الوقت خاصة على الأجهزة الضعيفة.	توزيعات لينكس مثل أوبونتو وآرتش تعمل بسلاسة حتى على الأجهزة ذات المواصفات المنخفضة.	الكفاءة Efficiency
يجمع بيانات المستخدمين ويحتوي على نظام تتبع وتحديثات تلقائية إلزامية.	لا يجبر المستخدم على إرسال بيانات أو تفعيل نظام تتبع كما في ويندوز.	الخصوصية Privacy

■ مكونات نظام التشغيل Linux

■ النواة (Kernel)

- النواة هي المكون الأساسي في نظام التشغيل لينكس، وهي الجزء الذي يتعامل مباشرة مع العتاد (المكونات المادية) للنظام وتدير الموارد الحيوية مثل:
 - إدارة الذاكرة (Memory management)
 - جدولة العمليات (Process Scheduling)
 - أنظمة الملفات (File Systems)
 - تعريفات الأجهزة (Device Drivers)
- توفر النوى الحديثة في لينكس (مثل الإصدار 6.x) أداءً محسناً، وقدرات زمن حقيقي (Real-Time Capabilities)، بالإضافة إلى دعم للأجهزة الحديثة الناشئة مثل معمارية RISC-V و مسرعات الذكاء الاصطناعي (AI Accelerators)

■ مساحة المستخدم في النظام (System User Space)

- تمثل هذه الطبقة العمليات على مستوى النظام، وتوفر أدوات للإعداد والأتمتة والتفاعل مع المستخدم. وتشمل ما يلي:
 - Shells: مثل Bash و Zsh ، وهي Command-line interfaces لتنفيذ الأوامر .
 - الخدمات الخلفية (Daemons): برامج تعمل في خلفية مثل الخدمات الشبكة، وتسجيل الأحداث (logging) و مراقبة النظام.
 - أنظمة التشغيل الأولي (init systems): مثل systemd، تدير عمليات الإقلاع و دورات حياة الخدمات.
 - بيئات سطح المكتب (Desktop Environments): مثل GNOME و KDE، وهي الواجهات الرسومية التي يتفاعل من خلالها المستخدم مع النظام.

■ التطبيقات (Applications)

- التطبيقات هي البرامج الموجهة للمستخدم التي تؤدي مهام محددة، وتتراوح بين:
 - أدوات الإنتاجية مثل LibreOffice و GIMP وهو تطبيق للرسم وتعديل الصور .
 - بيئات التطوير مثل VS Code و JetBrains IDEs .
 - برامج المؤسسات مثل أنظمة ERP وأدوات CRM.

تستخدم معظم توزيعات لينكس مديري الحزم (Package managers) مثل apt و dnf و pacman لتثبيت التطبيقات و تحديثها من مستودعات مركزية (Centralized Repositories).

■ ما هو ال shell

ال Shell هو برنامج (CLI - Command Line Interface) يعمل كجسر بين المستخدم والنواة (Kernel) .
يسمح ال Shell للمستخدمين — سواء كانوا مديريين للنظام أو مطورين بـ:

- إدخال الأوامر (تنزيل مكاتب , تحديث النظام)
- تنفيذ البرامج (فتح vscode)
- إدارة الملفات والعمليات (اضافة وحذف ملفات)
- التفاعل مباشرة مع نظام التشغيل

ملاحظة :

ال kernel بين المستخدم و العتاد الصلب

ال shell بين المستخدم و kernel

■ ما هو الامر Command

الأمر في لينكس هو برنامج أو أداة يتم تنفيذها عبر واجهة ال Command-Line حيث تعد هذه الواجهة واجهة نصية تتيح للمستخدمين التفاعل مباشرة مع نظام التشغيل من خلال كتابة التعليمات ثم تقوم ال Shell بترجمة هذه الأوامر ومعالجتها، ثم تتواصل مع النواة (Kernel) لتنفيذ المهام مثل إدارة الملفات، ومراقبة النظام، وإدارة البرمجيات.

■ بنية الأمر (Command)

جميع الأوامر لها البنية التالية: Command [Options] [Arguments]

Command	الكلمة التي تمثل الأمر, مثل: cd
Options	لتعديل سلوك الأمر عندما يكون ال option كلمة نسبه ب "--", مثل: "--help" عندما يكون حرفاً نسبه ب "-", مثل: "-n"
Arguments	اسم الملف أو أي بيانات يحتاجها الأمر ليقوم بعمله

يمكن أن يتواجد في بنية الأمر options و arguments معاً و ممكن أحدهما فقط ومن الممكن أن لا يتواجد اي منهما.

■ بعض الأوامر الأساسية

■ sudo (تعني "تشغيل الأمر بصلاحيات المدير" (Root))

• sudo apt update

هذا الأمر يخبر النظام عن أحدث الإصدارات للبرامج التي يمكن تثبيتها أو تحديثها. (لا يقوم بتحديث البرامج نفسها، بل فقط يُحدث قائمة الحزم).

• sudo apt install bash-completion

هذا الأمر يقوم بتنزيل وتثبيت الأداة bash-completion من مستودعات النظام.

بعد تثبيتها، ستصبح خاصية الإكمال التلقائي للأوامر في الشيل (Shell) متاحة، أي يمكنك الضغط على زر Tab لإكمال الأوامر أو أسماء الملفات تلقائياً.

■ --help

عند إضافة --help بعد أي أمر، يعرض النظام شرحاً مختصراً للأمر مع الخيارات والأمثلة، مثال: ls --help

■ man

يعرض صفحة تعليمات تفصيلية (Manual) لكل أمر، تشمل الوصف، الاستخدام، الخيارات، مثال: man ls

■ history

يمكنك رؤية قائمة بالأوامر السابقة (مع رقم لكل أمر)، وإعادة استخدامها بسهولة بدل كتابتها مرة أخرى . لإعادة تنفيذ أمر معين: !command-number

■ Whoami

ترجع اسم المستخدم الحالي

■ cd للتنقل بين المجلدات

أمثلة للاستخدامات:

• cd .. للعودة خطوة للوراء (المجلد السابق)

• cd / عودة لنقطة البداية /

• cd إذا قمنا بكتابة cd فقط يتم التوجيه للمجلد الموجود في /home للمستخدم الحالي

• cd path يمكن كتابة المسار كامل و الانتقال إليه

مثلاً نريد الانتقال للمجلد Music داخل مجلد المستخدم :rbcs

cd /home/rbcs/Music

■ ملاحظة :

(music != Music حساس لحالة الأحرف)

في لينكس يمكننا إنشاء ملفين مثلاً rbc.txt وآخر RBCS.txt بينما في ويندوز يعتبر أنه الملف نفسه.



- `cd folderName` إذا كان المجلد الذي نريد الانتقال إليه داخل المجلد الحالي يكفي كتابة اسمه فقط
مثلاً نحن في مجلد المستخدم `rbcs` ونريد الانتقال لمجلد `Music`: `cd Music`

■ `pwd` (print working directory) تقوم بعرض المسار الكامل للمجلد الحالي، مثل:
`/home/rbcs/Music`

■ `date`: يقوم بطباعة التاريخ و الوقت الحاليين على الشكل: Wed Nov 5 11:43:47 AM +03 2025

■ `cal`: يعرض التقويم (Calendar) للشهر الحالي

November 2025						
Su	Mo	Tu	We	Th	Fr	Sa
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

■ `uptime`: يعرض مدة عمل النظام منذ آخر تشغيل، مثلاً:

10:45:02up 2 hours, 15 min, 2 users, load average: 0.15, 0.10, 0.05

■ `whereis fileName`: يبحث عن مكان وجود برنامج أو ملف تنفيذي أو ملف المساعدة (man page) داخل النظام.

مثال: `ls: /bin/ls /usr/share/man/man1/ls.1.gz <= whereis ls`

■ `ls`: يعرض الملفات والمجلدات في الدليل الحالي.

تكون الملفات باللون الأبيض و المجلدات باللون الأزرق:

```
rbcs@ -VMware-Virtual-Platform:~/test$ ls
testdir testfile
```

■ `jobs`: يعرض قائمة بالعمليات (الأوامر) التي تعمل حالياً في ال shell الحالي حتى لو كانت عمليات تعمل في الخلفية (background processes) أو عمليات متوقفة (stopped processes)، إذا قمنا بفتح shell جديد فإن هذه العمليات لن تظهر هناك.

```
rbcs@ -VMware-Virtual-Platform:~$ jobs
[1]-  Running                  sleep 20 &
[2]+  Running                  sleep 20m &
```

- bg%n : إرسال المهمة للعمل في الخلفية (n هو رقم العملية في أمر job)
 - fg%n : يعيد المهمة لتعمل في الواجهة

■ sleep : يوقف تنفيذ الأوامر لفترة زمنية محددة تكون بالثواني بشكل افتراضي.

■ clear : يسمح محتوى الشاشة في ال terminal لتظهر فارغة ونظيفة.

■ echo : يُستخدم لعرض نص أو قيمة متغير على الشاشة .

```
rbcs@ -VMware-Virtual-Platform:~$ echo Hello World!
Hello World!
```

echo \$SHELL : يعرض نوع ال shell الذي يتم استخدامه حالياً

```
rbcs@ -VMware-Virtual-Platform:~$ echo $SHELL
/bin/bash
```

\$SHELL هو متغير في بيئة لينكس ويوجد متغيرات أخرى أيضاً مثل \$PWD وهو الذي يحوي على المسار الكامل للمجلد الحالي كما يمكن للمستخدم تعريف اي متغير اخر يريده واستخدامه

■ ادارة الملفات والمجلدات في لينكس

■ إنشاء مجلدات Folders:

- إنشاء مجلد: mkdir dirName يقوم بإنشاء مجلد داخل المجلد الحالي
- mkdir -p parentDir/childDir يقوم بإنشاء مجلدات داخل بعضها (تلقائياً) مرة واحدة.
- أي يقوم بإنشاء مجلد الأب أولاً parentDir ثم يقوم بإنشاء مجلد الابن بداخله childDir

■ إنشاء ملف (File):

touch filename يقوم بإنشاء ملف داخل المجلد الحالي مثلاً :
 touch project.py , touch rbc.txt , touch os.cpp

■ حذف مجلد/ملف :

- rm filename لحذف ملف
- rm -d dirName لحذف مجلد إذا كان فارغاً (إذا كان يحتوي على ملفات أو مجلدات أخرى يعطي خطأ)
- rm -r dirName لحذف مجلد مع محتوياته (وتقوم أيضاً بحذف ملف)
- يمكن حذف أكثر من ملف و مجلد في نفس الوقت: rm -r dirName1 dirName2 fileName

■ عرض محتويات ملف بالأسطر:

- more -n filename عرض المحتوى تدريجياً
- more filename : عرض محتوى الملف صفحة بصفحة (حجم الصفحة حسب حجم شاشة ال terminal)
- more -5 filename : عرض 5 أسطر من الملف في كل مرة.

• head/tail

- head filename: يعرض أول 10 أسطر من الملف.
- يمكن إضافة -n لتحديد عدد الأسطر head -5 filename
- tail: نفس حالات ال head لكنها تعرض الأسطر من نهاية الملف.

• cat

تقوم باستعراض محتويات الملف.

• cat > filename

يطلب إدخال النص الذي سيكون محتوى الملف، إذا لم يكن الملف موجود مسبقاً تقوم التعليمات بإنشائه وحفظ النص بداخله.

إذا كان الملف يحتوي بداخله على نص مسبقاً يتم حذفه و حفظ الجديد (override)

لإنهاء الإدخال: ctrl + D

• cat >> filename

نفس طريقة و حالات استخدام > لكن الفرق هنا إذا كان الملف يحتوي معلومات سابقاً لن يتم حذفها (append)

• cat < filename

تقوم بقراءة محتوى الملف وعرضه على الشاشة.

ملاحظة:

استخدام < يعني توجيه الإدخال (input redirection) من الملف إلى الأمر cat
بينما > يعني التوجيه إلى الملف أي الكتابة داخله .

■ النسخ و النقل:

• نسخ ملف:

cp path1/oldFileName path2/newFileName

تم نسخ الملف من المسار الأول للمسار الثاني، يمكن أن يتم تغيير اسمه عند النقل (newFileName)
إذا لم نرد تغيير الاسم نتركه كما هو.

➤ مثال:

لدينا المجلد test قمنا بإنشاء ملف بداخله testFile:

```
rbcs@ -VMware-Virtual-Platform:~/test$ touch testFile
rbcs@ -VMware-Virtual-Platform:~/test$ ls
testFile
```

نريد نقل هذا الملف للمجلد الفارغ test2:

```
rbcs@ -VMware-Virtual-Platform:~/test$ cp ./testFile /home/rbcs/test2
```

والان نتحقق من وجوده في test2 :

```
rbcs@ -VMware-Virtual-Platform:~/test2$ ls
testFile
```

- لنسخ مجلد نستخدم نفس التعليمة السابقة مع -r :

cp -r path1/oldDirName path2/newDirName

- نقل أو إعادة تسمية ملف أو مجلد:

mv oldName newName :إعادة تسمية مجلد/ملف: -

mv dirName fullPath :نقل مجلد: -

mv filename fullPath:نقل ملف: -

(استخدام ال -v مع التعليمة فقط لعرض تفاصيل عملية النقل / إعادة التسمية لكن التعليمة تعمل بدونها)

- تقسيم ملف كبير إلى عدة ملفات أصغر

- split -n N filename

تقسم الملف و تضع كل N محرف في ملف جديد

التسمية الافتراضية للملفات الجديدة: xaa xab xad

يمكن تغيير ال x في أسماء الملفات المنشأة عن طريق إضافة الكلمة المرادة بعد التعليمة مباشرة,

مثال: part_ split -n N filename تصبح أسماء الملفات : part_aa part_ab

- يمكن تقسيم الملف حسب الأسطر: split -l N filename

- البحث في المجلدات و الملفات:

- find

يستخدم للبحث عن الملفات أو المجلدات داخل النظام، اعتماداً على الاسم أو النوع أو التاريخ أو الموقع

الصيغة العامة للأمر:

[الملف أو الشرط] [الخيارات -] [التعبير الذي يحدد ما نبحث عنه] [المكان الذي نبدأ البحث منه] find

- `find /` : يعرض جميع الملفات والمجلدات الموجودة تحت المسار الجذري (أي من أعلى النظام)
- `find .` : يعرض كل الملفات والمجلدات الموجودة في المجلد الحالي فقط.
- `find /etc -name passwd` : يبحث في المجلد `/etc` عن أي ملف اسمه تمامًا `passwd`

```
rbc@ -VMware-Virtual-Platform:~$ find /etc -name passwd
find: '/etc/sssd': Permission denied
find: '/etc/credstore.encrypted': Permission denied
/etc/passwd
find: '/etc/cups/ssl': Permission denied
/etc/pam.d/passwd
find: '/etc/ppp/peers': Permission denied
find: '/etc/chatscripts': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/credstore': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
```

- `find . -name rbc -exec rm -r {} \;`
- يبحث في المجلد الحالي (.) عن ملف أو مجلد اسمه `rbc` ثم يقوم بحذفه عند العثور عليه.

شرح الرموز:

- `-exec` : تعني تنفيذ أمر معين على كل ملف يتم إيجاده.
- `{}` : يمثل الملف الذي تم العثور عليه (كل ملف/مجلد يتم إيجاده يطابق الاسم يتم تعويضه هنا ليتم تنفيذ الأمر عليه)
- `\;` : لإنهاء أمر التنفيذ.



• grep

يستخدم للبحث عن نصوص أو أنماط محددة داخل الملفات, يمكن استخدامه للبحث عن كلمات, جمل, أو أنماط باستخدام التعبيرات النمطية (regex).
الصيغة العامة للأمر:

grep [الملفات] pattern [خيارات]

أمثلة:

• grep root /etc/passwd

يبحث في الملف /etc/passwd عن أي سطر يحتوي على كلمة root

```
rbcs@ -VMware-Virtual-Platform:~$ grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
nm-openvpn:x:121:122:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
```

• grep ^root /etc/passwd

يبحث عن أي سطر يبدأ ب root

```
rbcs@ -VMware-Virtual-Platform:~$ grep ^root /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

• grep sh\$ /etc/passwd

يبحث عن أي سطر ينتهي ب sh

```
rbcs@ -VMware-Virtual-Platform:~$ grep sh$ /etc/passwd
root:x:0:0:root:/root:/bin/bash
tasneem:x:1000:1000:Tasneem:/home/tasneem:/bin/bash
user1:x:1001:1001:,,,:/home/user1:/bin/bash
rbcs:x:1002:1002:,,,:/home/rbcs:/bin/bash
```

• `grep r* /etc/passwd`

يبحث عن أي كلمة تحوي `r`

```
rbcs@ -VMware-Virtual-Platform:~$ grep r* /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-networkd:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesyncd:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
uidd:x:103:103::/run/uidd:/usr/sbin/nologin
usbmux:x:104:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tss:x:105:105:TPM software stack,,,:/var/lib/tpm:/bin/false
systemd-oom:x:990:990:systemd Userspace OOM Killer:/:/usr/sbin/nologin
kernoops:x:106:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
whoopsie:x:107:109::/nonexistent:/bin/false
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
```

• `grep r... /etc/passwd`

يبحث عن أي ورود لحرف ال `r` يليه 3 محارف (الفراغات تحتسب)

```
rbcs@ -VMware-Virtual-Platform:~$ grep r... /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-networkd:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesyncd:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
uidd:x:103:103::/run/uidd:/usr/sbin/nologin
usbmux:x:104:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tss:x:105:105:TPM software stack,,,:/var/lib/tpm:/bin/false
```

ملاحظة :

ما تم استخدامه مع grep مثل *, / , \$ والنقط هي بعض من تعابير regex يمكن كتابة تعابير أكثر تعقيدا مثلا للبحث عن كلمة تبدأ بحرف من بين أحرف معينة وتنتهي برقم وغير ذلك الكثير من التخصيص الذي نريده .

فائدة اريسيزية :

ما الفرق بين CLI, Terminal, Shell, cmd, powershell ؟

المصطلح	التعريف	النوع (type)	المنصة (platform)	الملاحظات/أمثلة
CLI	الطريقة العامة للتفاعل مع النظام باستخدام النصوص، وهي البيئة التي تكتب فيها الأوامر للتفاعل مع النظام.	نوع من واجهات التفاعل (Interface Type)	جميع الأنظمة (Windows, Linux, macOS)	وسيلة للتفاعل النصي مع النظام.
Terminal	الأداة أو النافذة التي تستخدم للوصول إلى الشيل وكتابة الأوامر فيها.	واجهة (Interface)	جميع الأنظمة	أمثلة: GNOME Terminal و xterm لليينكس، windows و terminal
Shell	مفسر الأوامر الذي يعمل داخل الطرفية (Terminal)، يقوم بقراءة الأوامر وتنفيذها.	برنامج (Program)	جميع الأنظمة	مثل Bash، Zsh، Fish.
Cmd	شيل تقليدي خاص بويندوز يُستخدم لتنفيذ أوامر نصية.	Shell	Windows فقط	أقدم من PowerShell وأقل قدرات.
Powershell	نوع من الشيل تم تطويره من مايكروسوفت، يُستخدم لإدارة الأنظمة وتنفيذ الأوامر النصية والبرمجية.	Shell	Windows، macOS، Linux	أقوى من CMD ويدعم البرمجة النصية Scripting

ملاحظة :

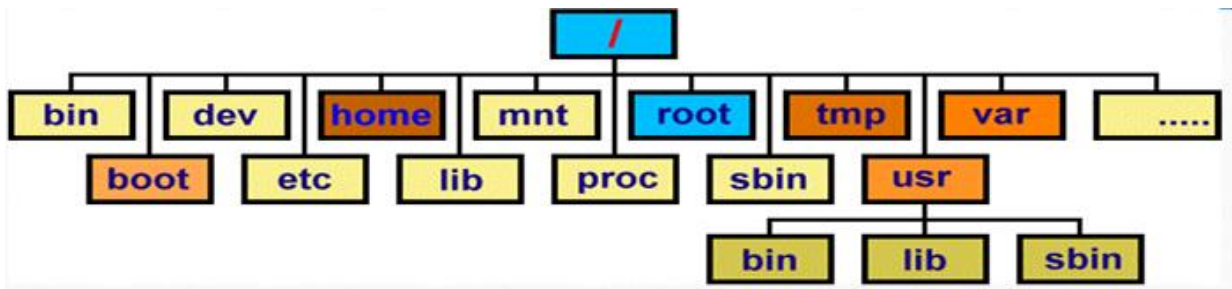
أوامر مثل `sudo apt update` لن تعمل على بعض توزيعات لينكس مثل Fedora لأنها تعتمد على مدير حزم (package manager) مختلف عن apt .

نظام الملفات في لينكس (Files System in Linux)

يشبه تخطيط نظام الملفات في لينكس إلى حد كبير التخطيط الموجود في الأنظمة الأخرى المشابهة لـ يونكس (Unix-like systems)

يعتمد تصميمه على معيار منشور رسمياً يُعرف باسم المعيار الهرمي لنظام ملفات لينكس

(Linux Filesystem Hierarchy Standard - FHS)، والذي يحدد كيفية تنظيم المجلدات والملفات في النظام ومكان تخزين كل نوع من البيانات (مثل ملفات النظام، المستخدمين، البرامج، والسجلات).



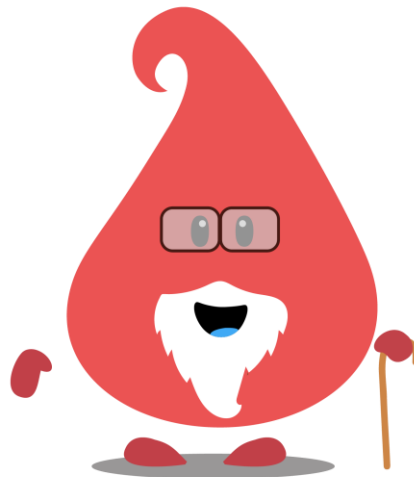
/	ال root لجميع المجلدات و الملفات الأخرى، كل مجلد أو ملف يبدأ ب /، يشبه ال C في ويندوز
bin(binary)	ملفات تشغيلية، تشبه ملفات ال .exe في windows
dev(device)	أي جهاز يتم وصله يتم إنشاء ملف له في هذا المجلد، مثل: mouse, keyboard,.....
home	كل مستخدم له مجلد خاص فيه ضمن هذا المجلد، كل مستخدم له مساحته و صلاحياته الخاصة (خاصية ال multiusers)
mnt (mount)	هو مجلد خاص يُستخدم لوضع (تركيب) الأقراص أو الأجهزة الخارجية مؤقتاً حتى يتمكن النظام من قراءتها والتعامل معها. فعندما نريد فتح قرص خارجي مثل USB أو قسم من القرص الصلب أو قرص من شبكة أخرى يدوياً، نقوم بتركيبه داخل هذا المجلد حتى تظهر ملفاته ويمكننا استخدامها. لو قمنا أيضاً بتنزيل نظام لينكس بجانب نظام ويندوز فإننا سنجد أقراص ويندوز داخله .

tmp (temporary)	جميع محتويات هذا الملف تحذف بعد فترة معينة.
var (variable)	مجلد ديناميكي، يخزن فيه كل ما يتغير حجمه مع الوقت، مثل: database, logs, .cache
boot	ملفات إقلاع.
etc	يحتوي على ملفات الإعداد (Configuration Files) وهي ملفات نصية (عادةً بصيغة عادية مثل conf أو بدون امتداد) تحتوي على الإعدادات والتعليمات التي يستخدمها النظام أو البرامج لتحديد كيفية عملها.
lib (library)	المكتبات التي نقوم بتنزيلها باستخدام apt مثلاً.
proc (process)	هذا المجلد له بنية هرمية أيضاً، يمكن اعتباره مشابه لل tasks في نظام ويندوز كمعنى عام و ليس حرفياً.
sbin	ملفات تشغيلية لكن للنظام (system binary).
usr (user)	كل ما يقوم المستخدم بالعمل عليه يوجد ضمن هذا المجلد، كما يحوي على مجلدات: bin, lib, sbin بداخله كي لا يقوم المستخدم بالتعديل على الملفات الأساسية للنظام.
root	المجلد الرئيسي لمستخدم Root (admin) صاحب أعلى صلاحيات في النظام.

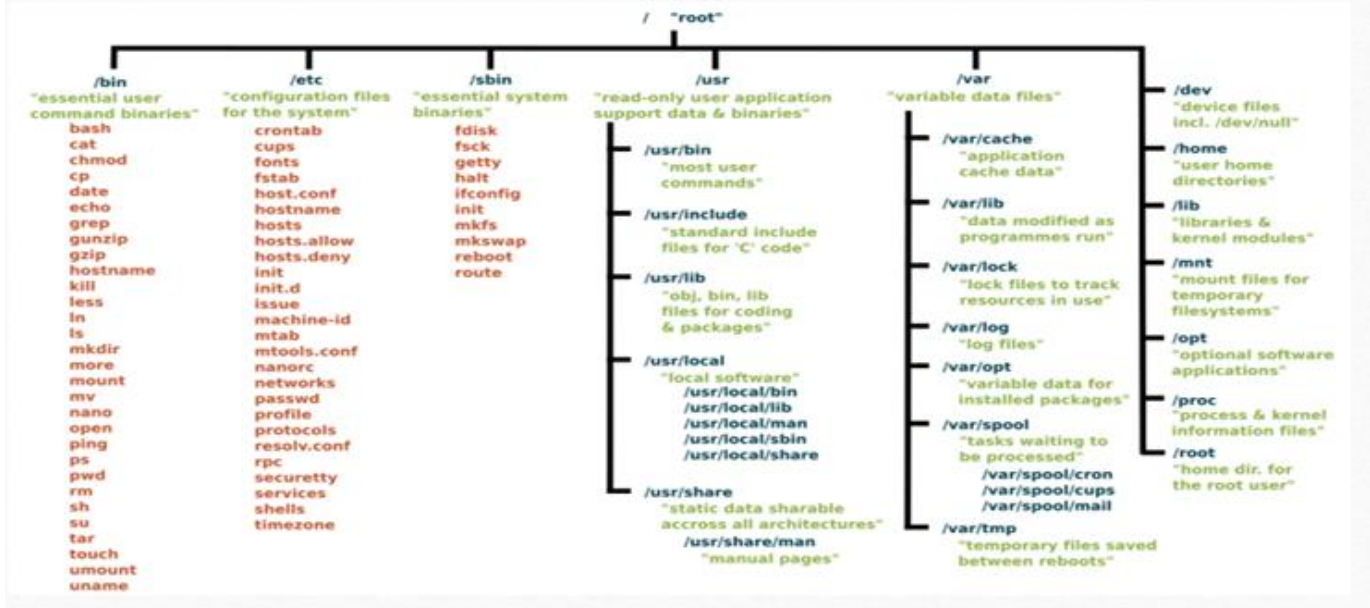
ملاحظة :

Every Thing in Linux is a File .

تعني هذه العبارة أن كل شيء في أنظمة تشغيل لينكس يتم التعرف عليه على أنه ملف فحتى أجهزة الإدخال والإخراج كما رأينا سابقاً مثل الكيبورد والماوس يعبر عنها بملفات والأقراص الصلبة كذلك .



Linux Filesystem Hierarchy



إعادة توجيه الإدخال والإخراج في لينكس (I/O Redirection)

- تسمح إعادة توجيه الإدخال والإخراج بتحديد من أين يأخذ الأمر بياناته (Input) وأين يرسل نتائج تشغيله (Output) حيث يمكن توجيه البيانات إلى: الشاشة (Terminal) , ملف, أمر آخر
- هذه الخاصية مفيدة جداً في:
التشغيل التلقائي (Automation)
كتابة السكريبتات (Scripting)

Stream	Descriptor	Description
stdin	0	Standard input (usually the keyboard)
stdout	1	Standard output (usually the screen)
stderr	2	Standard error (error messages)

إدارة تدفق البيانات في ال terminals

- عندما تكتب أمراً في الطرفية، يتم إدخال البيانات عبر stdin .
- النتائج تظهر عبر stdout .
- أخطاء التنفيذ تظهر عبر stderr .

أشهر رموز إعادة التوجيه (Common Redirection Operators)

الرمز	الوظيفة	المثال	الشرح
>	إعادة توجيه الإخراج إلى ملف (ويتم الكتابة فوقه)	ls > files.txt	يحفظ ناتج الأمر داخل الملف files.txt بدلاً من عرضها على الشاشة.
>>	إعادة توجيه الإخراج إلى ملف (مع الإضافة)	echo "new" >> log.txt	يضيف النص "new" إلى نهاية الملف log.txt دون مسح محتواه السابق.
<	إعادة توجيه الإدخال من ملف	sort < names.txt	يستخدم محتوى الملف names.txt كمدخل للأمر sort.
2>	إعادة توجيه رسائل الخطأ إلى ملف	cat test.txt 2> error.txt	باعتبار أن الملف test.txt غير موجود , يحفظ رسالة الخطأ الناتجة عن ذلك في الملف errors.txt
&>	إعادة توجيه الإخراج والأخطاء معاً إلى ملف واحد	./script.sh &> output.txt	يحفظ كلاً من ناتج التنفيذ وأي أخطاء في الملف output.txt
	تمرير ناتج أمر كمدخل لأمر آخر	cat file.txt grep "error"	بدلاً من كتابة grep file.txt "error"

نحاول عرض محتويات الملف test.txt لكنه غير موجود

```
rbcs@ -VMware-Virtual-Platform:~/RBCs$ cat test.txt
cat: test.txt: No such file or directory
rbcs@ -VMware-Virtual-Platform:~/RBCs$ ls
rbcs@ -VMware-Virtual-Platform:~/RBCs$ cat text.txt 2> error.txt
rbcs@ -VMware-Virtual-Platform:~/RBCs$ ls
error.txt
rbcs@ -VMware-Virtual-Platform:~/RBCs$ cat error.txt
cat: text.txt: No such file or directory
```

في التعليمية الأولى ظهر لنا خطأ عدم وجود الملف على الشاشة، و قمنا بالتأكد من عدم وجود الملف بتعليمية ls حيث أن المجلد فارغ.

في التعليمية الثالثة قمنا بتكرار التعليمية الأولى لكن هذه المرة قمنا بتوجيه خرج الخطأ للملف error.txt و لم يظهر الخطأ على الشاشة.

لاحظنا سابقاً أن الملف error.txt غير موجود في المجلد الحالي، لكن تم إنشاء ملف جديد بشكل تلقائي و محتوياته هي رسالة الخطأ .

مثال عن | :

لدينا ملف test.txt نريد البحث عن كلمة expert بداخله:

```
s@ -VMware-Virtual-Platform:~/RBCs$ cat test.txt | grep expe
expert in anything was once a beginner.
```



■ محرر نصوص VIM

Vim هو محرر نصوص تفاعلي يُستخدم لإنشاء الملفات النصية وتعديلها داخل ال terminal، بينما cat تستخدم فقط لعرض محتوى الملفات أو الإضافة عليها لكن دون إمكانية تعديل جزء محدد منها.

■ أهم الأوامر:

الأمر في Vim	الشرح
Esc	يبدّل من وضع الإدخال Input Mode إلى وضع الأوامر Command Mode يجب استخدامه قبل تنفيذ أي أمر.
i, a	يدخل إلى وضع الإدخال (الكتابة). i يكتب قبل موضع المؤشر. a يكتب بعد موضع المؤشر.
o	يفتح سطرًا جديدًا أسفل السطر الحالي ويدخل مباشرة في وضع الكتابة.
:wq	يحفظ (يكتب) الملف ويغلق Vim
:q!	يخرج من Vim بدون حفظ التغييرات.
:w filename	يحفظ الملف باسم جديد (ينشئ ملفاً جديداً بنفس المحتوى).
dd	يحذف السطر الحالي.
yy	ينسخ السطر الحالي.
P	يلصق (Paste) النص المنسوخ أو المحذوف بعد موضع المؤشر.
V	يدخل إلى وضع التحديد Visual Mode لتحديد النصوص (يمكن بعدها نسخها أو حذفها).
u	يتراجع (Undo) عن آخر تعديل. يمكن تكراره للتراجع عن تعديلات أقدم.
Ctrl + r	يعيد (Redo) آخر عملية تراجع عكس الأمر u
gg	ينقل المؤشر إلى أول سطر في الملف.
G	ينقل المؤشر إلى آخر سطر في الملف

ملاحظة :

(!) تعني "إجبار الخروج" بدون تأكيد .

تمرين اريسيزي :

لنقم بإنشاء ملف باسم rbc.txt ونسند إليه عبارة linux is great operating system
 ثم قم بإنشاء ملف آخر test.txt وضع داخله السجل الخاص بالترمينال (history)
 أضف محتوى test.txt لنهاية rbc.txt
 أبحث عن عبارة rbc في الملف
 عرف متغير جديد واسند إليه نتيجة البحث عن rbc
 الحل :

```
abd@abd:~/Desktop$ touch rbc.txt
abd@abd:~/Desktop$ "linux is great operating system " > rbc.txt
linux is great operating system : command not found
abd@abd:~/Desktop$ echo "linux is great operating system " > rbc.txt
abd@abd:~/Desktop$ touch test.txt
abd@abd:~/Desktop$ history > test.txt
abd@abd:~/Desktop$ cat test.txt >> rbc.txt
abd@abd:~/Desktop$ echo "rbc 90A" >> rbc.txt
abd@abd:~/Desktop$ grep rbc rbc.txt
1089 touch rbc.txt
1090 "linux is great operating system " > rbc.txt
1091 echo "linux is great operating system " > rbc.txt
rbc 90A
abd@abd:~/Desktop$ VARIABLE=$(grep "rbc" rbc.txt)
abd@abd:~/Desktop$ echo $VARIABLE
1089 touch rbc.txt 1090 "linux is great operating system " > rbc.txt 1091 echo
"linux is great operating system " > rbc.txt rbc 90A
abd@abd:~/Desktop$ echo $VARIABLE >> test.txt
abd@abd:~/Desktop$
```

الشرح :

- الأمر الأول : قمنا بإنشاء ملف rbc.txt باستخدام الأمر .touch
- الأمر الثاني : حاولنا إسناد القيمة النصية بدون استخدام echo وهذا يعطي خطأ فلترمينال يحاول التعرف عليه على انه أمر ولا يجده .
- الأمر الثالث : إسناد بشكل صحيح عن طريق echo .

- الأمر الرابع : إنشاء ملف test.txt.

- الأمر الخامس : قمنا باستخدام الأمر history وحده سيقوم بطباعة سجل الأوامر على الترمinal ولكننا لم نستخدمه لوحده بل قمنا بإسناد قيمة الخرج الصادرة من هذا الأمر إلى الملف test.txt.

فأصبح الملف يحوي بداخله سجل الأوامر التي تمت كتابتها.

- الأمر السادس : قمنا باستخراج محتوى الملف test.txt عن طريق الأمر cat ومن ثم أسندناه الى نهاية الملف rbc.txt.

لماذا الأمر الثاني لم يعمل بدون echo ولكن في الأمر الخامس عمل بشكل صحيح باستخدام history وحده بينما في الأمر السادس احتجنا الأمر cat :

الأمر الثاني : دعنا نأخذ "linux is great operating system" لوحدها تنفيذها يعطي خطأ اي أنها لا تعطي خرج على الترمinal .

اما الأمر الخامس : history فإنه يعطي قيمة بالفعل لذا يمكننا استخدامها مباشرة الأمر ذاته ينطبق ايضاً على أوامر مثل ls وغيرها .

بالنسبة للأمر السادس : فإن كتابة test.txt >> rbc.txt خاطئة لأن الترمinal سيحاول التعرف على test على أنها أمر كما أنها لا تعطي أي خرج لذا نحتاج للأمر cat لاستخراج محتوى الملف ثم نقوم باستخدامه.

- الأمر السابع : قمنا بإضافة عبارة تحوي على rbc وذلك كي تظهر عند البحث لاحقاً.

- الأمر الثامن : قمنا بالبحث عن العبارة rbc .

لاحظ الخرج يحوي على :

Touch rbc.txt هذه العبارة أصبحت ضمن الملف عندما قمنا بنسخ محتوى ملف test إلى الملف rbc

حيث أن هذا الملف يحتوي على السجل أي الأوامر السابقة (ما ظهر هو الأمر الذي كتب في الترمinal ثم حفظ في السجل ثم نقل إلى الملف و الأمر ذاته بالنسبة لبقية نتائج البحث كلها من السجل عدا آخر سطر هو الذي قمنا بإسناده بأنفسنا في الأمر السابع)

- الأمر التاسع والعاشر :

قمنا بتعريف متغير جديد يدعى VARIABLE ووضعنا بداخله قيمة البحث .

ثم قمنا بطباعة المتغير للتأكد فحصلنا بالفعل على نتيجة البحث السابقة .

ملاحظة على المتغيرات :

عند تعريف متغير جديد لا نضع إشارة ال \$ ولكن عندما نريد استخدامه لاحقاً نضعها.
لإسناد قيمة للمتغير نستخدم العبارة \$() ونضع داخل الأقواس أي شيء يعطي خرج مثل "hello" echo
"world" على سبيل المثال.
يمكن تغيير قيمة المتغير لاحقاً أي أنها ليست ثابتة .

- الأمر الحادي عشر :

المتغير بداخله قيمة نصية أي يمكننا كذلك إضافة قيمته لنهاية الملف واستخدامه بحرية لاحظ قمنا باستخدام echo
لمستخراج قيمة المتغير وإلا سيحدث نفس الخطأ الذي حدث في الأمر الثاني.

أنتهت المحاضرة....

